

Componentes, Props e States

Objetivo da Aula

O principal objetivo da aula é capacitar o aluno a compreender o conceito de componentização no React, construir componentes funcionais e baseados em classe, identificar as suas diferenças, aplicar props para personalizar e transmitir dados entre componentes, bem como compreender o gerenciamento de states para criar interfaces interativas e reativas, seguindo boas práticas de desenvolvimento.

Apresentação

Seja bem-vindo à aula de **componentes, props e states** no React, uma das bases fundamentais para o desenvolvimento de aplicações front-end dinâmicas e interativas. Nesta jornada de aprendizado, nosso objetivo é proporcionar uma compreensão sólida dos conceitos de componentização, propriedades (props) e gerenciamento de estado no contexto do React.

Iniciaremos entendendo o conceito de componentes React, começando com as diferenças entre componentes funcionais e baseados em classe. Compreender a estrutura desses componentes é essencial para criar uma arquitetura robusta e escalável.

Em seguida, exploraremos como as props podem ser passadas aos componentes e propagadas para demais componentes.

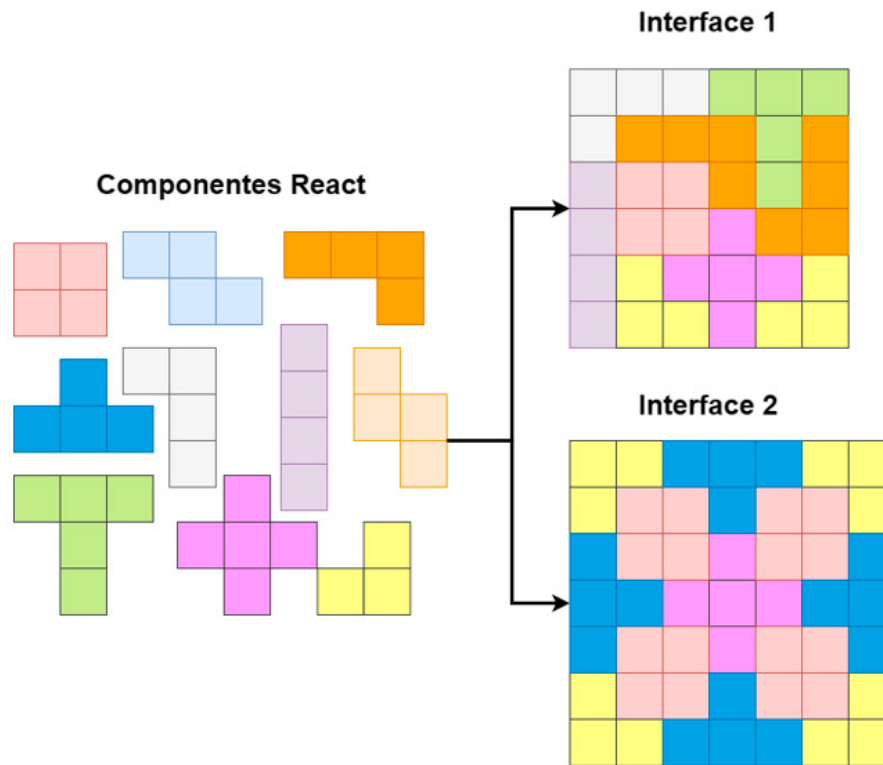
Por fim, mergulharemos no gerenciamento de estado dos componentes necessários para controlar mudanças e atualizações na interface. Veremos como criar estados e modificar seus valores e, ainda, como esses conceitos estão intimamente relacionados com a reatividade das aplicações React.

1. Componentes React

Conforme Minnick (2022), uma das principais características do React é a capacidade de criar componentes reutilizáveis. Os componentes React são blocos de construção

fundamentais da interface do usuário do React. Eles podem ser combinados para compor diferentes interfaces e estruturar toda a aplicação. A Figura 1 ilustra componentes React sendo reutilizados para compor as interfaces 1 e 2.

Figura 1: Composição de interfaces utilizando componentes React



Fonte: Elaboração própria.

O React permite que você possa declarar componentes baseados em classes e em funções. Nas próximas seções, você aprenderá na prática como construir componentes React baseados em classe e em funções. Ao longo dos exemplos, você aprenderá os conceitos, de forma progressiva, compreendendo, inicialmente, como construir um componente básico baseado em classes e em funções. Em seguida, aprenderá a utilizar propriedades e, por fim, aprenderá a gerenciar o estado dos componentes.



Destaque

De acordo com a nova documentação do React (React.Dev, 2023), embora componentes de classes ainda sejam suportados pelo React, **é recomendado descontinuar o uso de classes e migrar para componentes funcionais.**

2. Criando o seu Primeiro Componente

Para dar sequência ao seu aprendizado e fixar os conceitos a respeito de componentes, crie um projeto React por meio do <https://codesandbox.io> ou, localmente, através do terminal, utilizando o comando ***npx create-react-app criando-componentes***.

2.1. Estrutura de Diretórios

Ao criar o seu novo projeto, você terá uma estrutura de diretórios da aplicação. Os principais arquivos dessa estrutura são apresentados a seguir, contendo as pastas **public** e **src**.

```
criando-componentes
├── package.json
├── public
│   └── index.html
└── src
    ├── App.js
    └── index.js
```

2.2. Criando Componentes Baseados em Classes e em Funções

Após a construção inicial do seu projeto, crie, dentro do diretório **src**, a pasta **components**. Em **components**, você irá criar o seu primeiro componente de classe, **BotaoClasse.jsx**, e de função, **BotaoFuncao.jsx**, conforme os códigos apresentados em seguida.

BotaoClasse.jsx

```
import { Component } from 'react';
export default class BotaoClasse
extends Component {
  render() {
    return (
      <button type='button'
```

```
    onClick={() =>
      console.log('Clicou!')}
  >
  Clique (Classe)
</button>
)
}
}
```

BotaoFuncao.jsx

```
export default function
BotaoFuncao() {
  return (
    <button type='button'
      onClick={() =>
        console.log('Clicou!')}
    >
    Clique (Função)
    </button>
  )
}
```

Similar ao componente de classe, o componente funcional opera de forma idêntica ao método `render()`.

Em seguida, você terá a seguinte estrutura de diretórios.

```
criando-componentes
├─ package.json
├─ public
```

```

|   └─ index.html
└─ src
    └─ components
        └─ BotaoClasse.jsx
        └─ BotaoFuncao.jsx
└─ App.js
└─ index.js

```



Destaque

Em projetos React, é de boas práticas a utilização da pasta **components** para a escrita dos componentes da interface.



Destaque

A pasta **public** é um diretório, cujos arquivos são visíveis pela url do navegador, enquanto a pasta **src** é um diretório que contém a lógica da aplicação cujo código não pode ser acessado diretamente do navegador.

Para adicionar os botões à sua aplicação, altere o arquivo **src/App.js**, conforme o código a seguir.

```

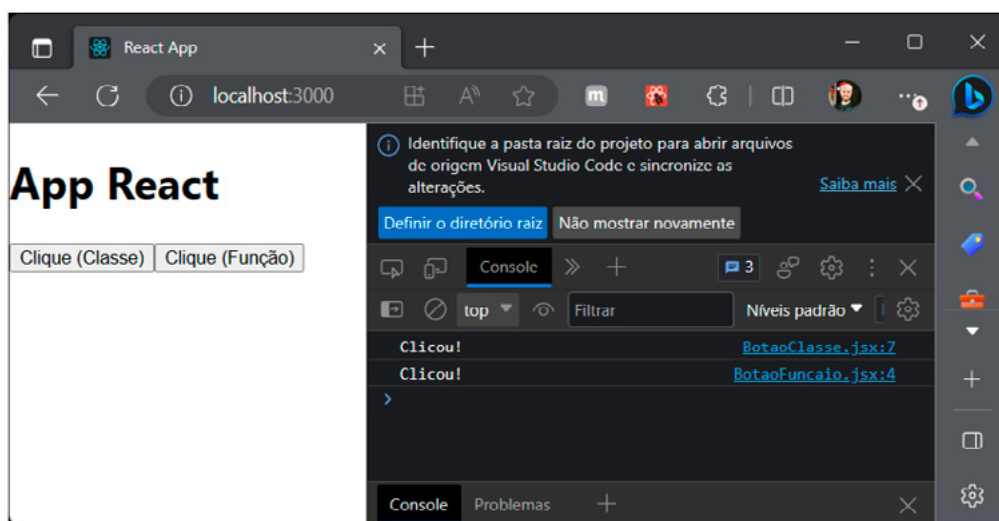
import BotaoClasse from './components/BotaoClasse';
import BotaoFuncao from './components/BotaoFuncao';
export default function App() {
  return (
    <div>
      <h1>App React</h1>
      <BotaoClasse />
      <BotaoFuncao />
    </div>
  );
}

```

Observe que os componentes foram importados com os nomes **BotaoClasse** e **BotaoFuncao**. É importante que os componentes sejam definidos com a letra maiúscula, pois, na sintaxe JSX, as tags maiúsculas são entendidas como componentes, enquanto as minúsculas como elementos HTML.

Por fim, execute a sua aplicação, com o comando no terminal **npm start**, e acesse o endereço <https://localhost:3000>. A interface resultante contém os componentes de classe e de função, conforme a Figura 2. Esses componentes são equivalentes e contém a mesma lógica. Ao clicar no botão, será apresentada, no console do navegador, a mensagem "Clicou!".

Figura 2: Página resultante, com os componentes de classe e funcional



Fonte: Elaboração própria.

3. Utilizando Propriedades dos Componentes

As propriedades dos componentes, também conhecidas como **props**, são utilizadas para passar dados de entrada aos componentes. As props são passadas para os componentes como atributos HTML e podem ser acessadas pelo componente como uma propriedade do objeto **props**.



Destaque

As props podem receber valores de diferentes tipos de dados, desde strings, listas e objetos até funções.

3.1. Passando Props aos Componentes

Dando sequência aos componentes criados na Seção 2.2., iremos adicionar a propriedade nome aos componentes **BotaoClasse** e **BotaoFuncao**, no arquivo `/src/App.js`, conforme o código apresentado a seguir.

```
import BotaoClasse from './components/BotaoClasse';
import BotaoFuncao from './components/BotaoFuncao';
export default function App() {
  return (
    <div>
      <h1>App React</h1>
      <BotaoClasse nome="Botão Classe" />
      <BotaoFuncao nome="Botão Função" />
    </div>
  );
}
```

Como a sintaxe JSX não faz distinções entre componentes de classe ou função, também não há distinções na passagem de **props** aos componentes.

3.2. Acessando Props dos Componentes

As props são acessadas de formas diferentes em componentes funcionais e baseados em classe. Em componentes funcionais, props é um parâmetro passado à função do componente, enquanto em classes, props é herdado da classe **Component** e acessada pelo objeto com *this.props*.

Os códigos a seguir incluem alterações de como as props podem ser acessadas em componentes de classe e de funções.

BotaoClasse.jsx

```
import { Component } from 'react';
export default class BotaoClasse extends Component {
  render() {
    return (
      <button type='button'
        onClick={() =>
          console.log('Clicou!')}
      >
        {this.props.nome}
      </button>
    )
  }
}
```

BotãoFunção.jsx

```
export default function
BotaoFuncao(props) {
  return (
    <button type='button'
      onClick={() =>
        console.log('Clicou!')}
    >
      {props.nome}
    </button>
  )
}
```

3.3. Utilizando a Props Children

Em react, **children** é uma prop especial que permite que você passe elementos-filho aos componentes. Isso é útil quando é preciso adicionar conteúdo extra aos componentes, sem que você tenha que modificá-los em seu código diretamente. A seguir, temos um exemplo de uso da propriedade **children** em componentes de classe e função.

Exemplo utilizando a classe.

```
import { Component } from 'react';  
export default class Cabecalho extends Component {  
  render() {  
    return (  
      <header>  
        <h1>App React</h1>  
        {this.props.children}  
      </header>  
    )  
  }  
}
```

Exemplo utilizando função.

```
export default function  
  Cabecalho(props) {  
    return (  
      <header>  
        <h1>App React</h1>  
        {props.children}  
      </header>  
    )  
  }
```

Independentemente de você estar utilizando componentes de classe ou de função, aqui está um exemplo de como você pode passar elementos-filho para o componente **Cabecalho**, utilizando a props *children*.

```
export default function App() {  
  return (  
    <div>  
      <Cabecalho />  
      <Cabecalho>  
        <p>Descrição</p>  
      </Cabecalho>  
      <Cabecalho>  
        <h2>Subtítulo</h2>  
        <p>Outra descrição</p>  
      </Cabecalho>  
    </div>  
  );  
}
```

Com base nesse código, o primeiro componente **Cabecalho** irá renderizar apenas o título “App React”, uma vez que nenhum elemento-filho foi passado ao componente. Por sua vez, o segundo **Cabecalho** irá renderizar o título “App React” e o parágrafo “Descrição”. Por último, o terceiro **Cabecalho** irá introduzir dois elementos-filho, renderizando o título “App React”, o subtítulo “Subtítulo”, além do parágrafo “Outra descrição”.

4. Gerenciamento de Estado dos Componentes

Através do paradigma orientado a objetos (POO), cada objeto de uma classe tem o controle do seu estado interno, representado por atributos, cujos métodos (ou ações) podem refletir em mudanças de estado (Sebesta, 2018). Entretanto, assim como *props*, componentes baseados em classe declaram o estado do componente através do atributo state, herdado da classe **Component**. Esse estado só pode ser alterado ao utilizar o método `setState()`.

Por outro lado, para acessar o estado atual do componente, o conteúdo deverá utilizar o `this.state`, para acessar estado, e `this.setState()`, para alterar o estado do componente. Em contrapartida, componentes funcionais não possuem nativamente estado, sendo necessária a utilização do **hook `useState`** para o gerenciamento de estado do componente.

A seguir, os códigos apresentados em **BotaoClasse.jsx** e **BotaoFuncao.jsx** incluem um exemplo de gerenciamento de estado aos componentes de classe e funcionais, respectivamente.

BotaoClasse.jsx

```
import { Component } from 'react';
export default class BotaoClasse extends Component {
  constructor(props) {
    super(props);
    this.state = { contador: 0 };
  }
  incrementa = () => {
    this.setState({ contador: this.state.contador + 1 });
  }
  render() {
    return (
      <button onClick={this.incrementa}>
        Cliques: {this.state.contador}
      </button>
    );
  }
}
```

BotaoFuncao.jsx

```
import { useState } from 'react'

export default function BotaoFuncao() {

  const [contador, setContador] = useState(0);

  function incrementa() {

    setContador(contador + 1);

  }

  return (

    <button type='button' onClick={incrementa}>

      Cliques (Função): {contador}

    </button>

  )

}
```

Em **componentes de classe**, o valor do atributo **state** deve ser do tipo **Object**. Se você deseja gerenciar o estado de várias propriedades de seu componente, essas propriedades deverão ser definidas em um único objeto. O estado inicial do componente é definido em sua inicialização, dentro do construtor, como ocorre em **this.state = { contador: 0 }**.

Em componentes de classe, sempre que definir métodos que reagem a eventos, é recomendado que você utilize **arrow functions**. Caso contrário, você encontrará problemas se definir o método conforme o código abaixo.

```
incrementa() {

  this.setState({ contador: this.state.contador + 1 });

}
```

Ao realizar o clique no botão, você encontrará a seguinte mensagem de erro: **TypeError: Cannot read properties of undefined (reading 'setState')**. Para resolver esse problema, basta incluir a seguinte operação no construtor: **this.increment = this.increment.bind(this)**.

Por outro lado, em **componentes funcionais**, o *hook* `useState` retorna uma lista contendo dois valores: o valor do estado atual e uma função de atualização desse estado, como ocorre em `const [contador, setContador] = useState(0)`. Essa sintaxe utiliza a técnica de desconstrução para inicializar as variáveis `contador`, com o valor do estado atual, inicializado com `0`, e `setContador`, com a função de atualização de estado.



Destaque

Em componentes funcionais, você não precisa centralizar todos os estados do componente em uma variável do tipo `Object`; você pode declarar quantos estados quiser, com o *hook* `useState`, separadamente – o que não ocorre em componentes de classe.

Em contraste aos componentes de classe, os componentes funcionais não apresentam problemas ao utilizar funções que reagem a eventos da interface.

Considerações Finais da Aula

Ao final desta aula, podemos afirmar que você já está equipado com conhecimentos essenciais para construir aplicações front-end interativas e dinâmicas.

Aprendemos a construir componentes funcionais e baseados em classe, além de como eles se tornam os blocos de construção fundamentais na construção de interfaces. Com a capacidade de criar componentes reutilizáveis, podemos desenvolver aplicações complexas de maneira mais eficiente e organizada.

Exploramos, ainda, a fundo o conceito de **props** e como eles permitem que os componentes recebam dados externos e propaguem esses dados a outros componentes. Isso nos possibilita criar componentes flexíveis e customizáveis, adaptando-os a diferentes necessidades, sem alterar sua lógica interna.

Por fim, com o gerenciamento de estado dos componentes, aprendemos a lidar com informações que mudam ao longo do tempo. O uso do estado nos possibilita criar interfaces reativas, em que as atualizações são feitas automaticamente, melhorando a experiência do usuário.

Ao concluir esta aula, tenha em mente que os conceitos de componentes, **props** e **states** são essenciais para o desenvolvimento de um React de alto nível.

Material Complementar



Component

2023, React.

A documentação oficial do React não recomenda mais a utilização de componentes de classe em códigos futuros. Nesse site, você poderá aprender como migrar de componentes baseados em classe para componentes funcionais, dando a você uma boa visibilidade das diferenças entre os componentes. É importante saber essas diferenças para minimizar eventuais confusões de compreensão entre os conceitos.

Link para acesso: <https://react.dev/reference/react/Component#alternatives> (acesso em 18 set. 2023.)

Referências

CODESANDBOX. Disponível em: <https://codesandbox.io>. Acesso em: 1. jul. 2023.

MINNICK, Chris. **Beginning ReactJS Foundations Building User Interfaces with ReactJS**: An Approachable Guide. ed. 1. Nova Jersey: Wiley, 2022.

REACT. **Component**. React, 2023. Disponível em: <https://react.dev/reference/react/Component>. Acesso em: 1 jul. 2023.

SEBESTA, Robert W. **Conceitos de Linguagens de Programação**. ed. 11. Porto Alegre: Bookman, 2018.